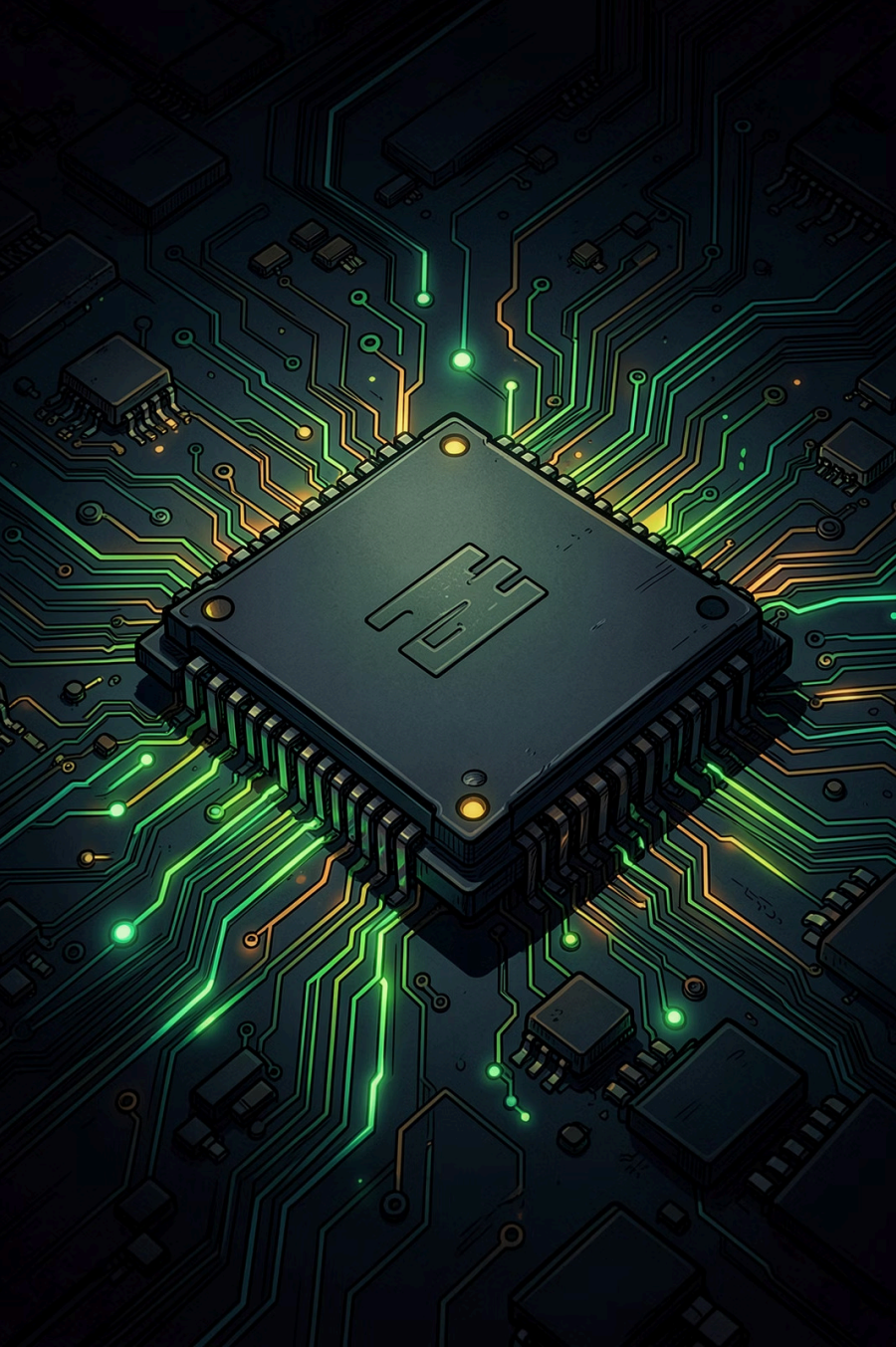




Simulador de Escalonamento de CPU

PUC Minas — Sistemas Operacionais 2026/1 | Trabalho Prático 1

Nomes: Gustavo Carvalho Ferreira Pinheiro, Lucas Alexandre Soares Gomes da Silva,
Lucas Guimarães Pós, Pedro Soares de Souza Garcia

Visão Geral – Padrão Strategy

O Contrato Central

A interface `Scheduler` define um único contrato: `run(List<Process>) → Metrics`. Todos os algoritmos implementam essa interface. O `Main` não sabe — e não precisa saber — qual algoritmo está rodando.

Fluxo de Execução

O `Main` lê o arquivo via `FileParser`, cria uma cópia limpa dos processos para cada algoritmo com `Process.copy()`, chama `run()` e recebe um objeto `Metrics`. Ao final, imprime a tabela comparativa.

Main

- └ lê arquivo → `FileParser`
- └ para cada algoritmo:
 - └ copia processos → `Process.copy()`
 - └ roda → `Scheduler.run()`
 - └ recebe → `Metrics`
 - └ imprime tabela comparativa

As Classes e Suas Responsabilidades

Process

Representa um processo do SO. Armazena dados fixos (PID, burst, chegada, I/O) e estado dinâmico (burst restante, CPU acumulada, tau). O método `copy()` isola cada simulação.

Scheduler

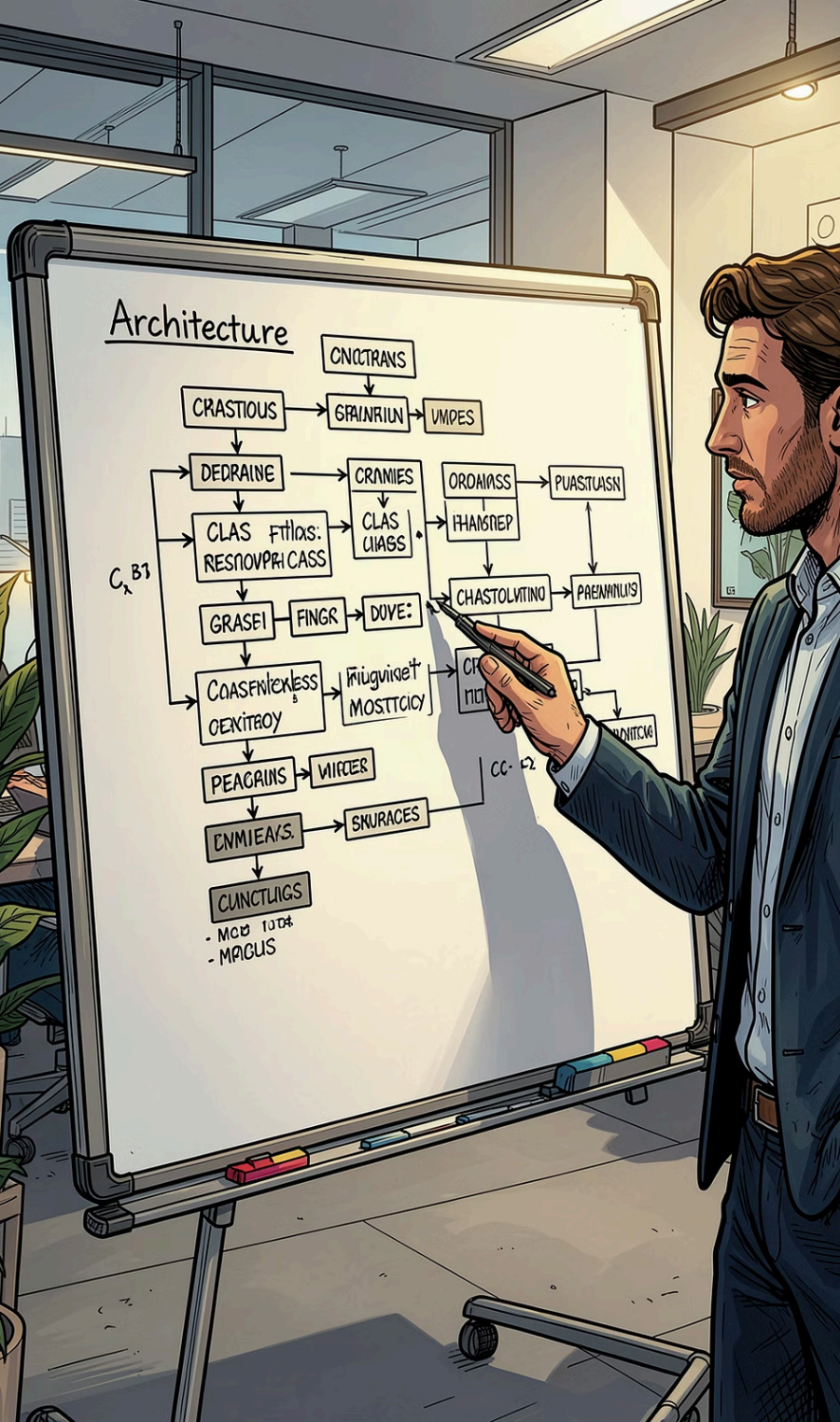
Interface com `run()`. Define o contrato compartilhado por todos os algoritmos de escalonamento.

FileParser

Lê `processos.txt` e transforma cada linha em um objeto `Process`. Separado do `Main` pelo princípio de responsabilidade única.

Metrics

Encapsula os resultados: espera média, turnaround médio e throughput. Calculados uma vez e expostos via getters.



Como a Simulação Funciona

Todos os algoritmos simulam o tempo **tick a tick** (`currentTime++`). A cada tick, cinco etapas são executadas em ordem:

1

Libera I/O

Processos que voltam do I/O são liberados para a fila de prontos.

2

Admite Chegadas

Processos com `arrivalTime ≤ currentTime` entram na fila.

3

Decide quem roda

← Aqui cada algoritmo difere. É o coração da estratégia.

4

Executa 1 tick de CPU

Uma unidade de CPU é executada no processo escolhido.

5

Verifica I/O ou Fim

Checa se o processo acionou I/O ou foi concluído.

Os Quatro Algoritmos

FCFS

Primeiro da fila por ordem de chegada.

NÃO PREEMPTIVO

SRTF

Menor burst restante; preempta se chegar processo mais curto.

PREEMPTIVO

Round-Robin

Fila circular; quantum = menor tau entre processos prontos.

PREEMPTIVO

MLQ

Fila 1 (alta prioridade, RR) tem prioridade absoluta sobre fila 2 (FCFS).

ENTRE FILAS

Comparativo dos Algoritmos

Algoritmo	Lógica de Decisão	Preemptivo?
FCFS	Primeiro da fila por chegada	Não
SRTF	Menor burst restante; preempta se chegar alguém mais curto	Sim
Round-Robin	Fila circular; quantum = menor tau entre processos prontos	Sim
MLQ	Fila 1 (prio alta, RR) tem prioridade absoluta sobre fila 2 (FCFS)	Entre filas

A diferença entre os quatro algoritmos está **exclusivamente na etapa de decisão** do tick — todo o restante da simulação é compartilhado.

Resumo do Projeto



Padrão Strategy

Interface `Scheduler` unifica todos os algoritmos sob um único contrato `run() → Metrics`.



Isolamento com `copy()`

`Process.copy()` garante que cada algoritmo receba um estado limpo e independente.



Simulação por Ticks

Cada tick executa 5 etapas fixas; apenas a decisão de escalonamento varia por algoritmo.



Métricas Comparativas

Espera média, turnaround médio e throughput calculados e exibidos em tabela comparativa ao final.